cka_step_by_step

October 6, 2026

1 CKA paso a paso — matrices chiquititas para entender qué mide

Mismo espíritu que la exploración de effective rank del principio: en vez de confiar en `modules/cka.py` como caja negra, aquí construimos matrices X, Y de juguete (pocos ejemplos, pocas features, números que se pueden seguir a mano) y recalculamos CADA paso de la fórmula por separado, para ver exactamente qué se multiplica y por qué el resultado sale como sale.

Fórmula (`linear_cka`, Kornblith et al. 2019, kernel lineal):

$$\mathrm{CKA}(X, Y) = \frac{\|Y^\top X\|_F^2}{\|X^\top X\|_F \cdot \|Y^\top Y\|_F}$$

con $X \in \mathbb{R}^{n \times p_1}$, $Y \in \mathbb{R}^{n \times p_2}$ — n ejemplos, columnas centradas (media 0 por feature) antes de todo. Dan un escalar en $[0, 1]$: 1 si las representaciones son idénticas salvo rotación/reflexión/permutación de features y escala isotrópica; cerca de 0 si no comparten estructura.

[1]:
```
import sys
from pathlib import Path

import numpy as np
import torch

project_root = Path.cwd().resolve().parent
sys.path.append(str(project_root))

from modules.cka import linear_cka

np.set_printoptions(precision=3, suppress=True)
torch.set_printoptions(precision=3, sci_mode=False)
```

1.1 La función "debug" — recalcula cada paso a mano

Nada aquí llama a `linear_cka` primero; se recalcula todo con numpy/pytorch explícito, paso por paso, y al final se compara contra `linear_cka` para confirmar que coinciden. Si en algún momento no coinciden, es la señal de que algo en el entendimiento está mal — esa es literalmente la función de este notebook.

[2]:
```
def cka_step_by_step(X, Y, label=""):
    X = torch.as_tensor(X, dtype=torch.float64)
    Y = torch.as_tensor(Y, dtype=torch.float64)

    print(f"=== {label} ===")
    print(f"X: {tuple(X.shape)} (n={X.shape[0]} ejemplos, p1={X.shape[1]}␣
↪features)")
    print(f"Y: {tuple(Y.shape)} (n={Y.shape[0]} ejemplos, p2={Y.shape[1]}␣
↪features)")

    # 1) Centrar columnas (media 0 por feature) -- CKA no le importa el offset/
↪bias
    X_c = X - X.mean(dim=0, keepdim=True)
    Y_c = Y - Y.mean(dim=0, keepdim=True)
    print("\nX centrada:\n", X_c.numpy())
    print("Y centrada:\n", Y_c.numpy())

    # 2) El término cruzado Y^T X: (p2, n) x (n, p1) -> (p2, p1)
    #    Cada entrada [a, b] es el producto punto entre la feature 'a' de Y y la
    #    feature 'b' de X, a lo largo de los n ejemplos -- qué tanto "se mueven␣
↪juntas".
    cross = Y_c.T @ X_c
    print(f"\nY_c^T @ X_c  (shape {tuple(cross.shape)} = p2 x p1):\n", cross.
↪numpy())

    cross_term = torch.linalg.matrix_norm(cross) ** 2
    print(f"\n||Y_c^T X_c||_F^2 = {cross_term.item():.6f}")

    # 3) Las normas "propias" de cada representación -- cuánta estructura␣
↪interna tiene
    #    cada una por separado (el denominador normaliza para que el resultado␣
↪quede en [0,1])
    xtx = X_c.T @ X_c
    yty = Y_c.T @ Y_c
    xtx_norm = torch.linalg.matrix_norm(xtx)
    yty_norm = torch.linalg.matrix_norm(yty)
    print(f"\nX_c^T @ X_c:\n{xtx.numpy()}\n||X_c^T X_c||_F = {xtx_norm.item():.
↪6f}")
    print(f"\nY_c^T @ Y_c:\n{yty.numpy()}\n||Y_c^T Y_c||_F = {yty_norm.item():.
↪6f}")

    cka_manual = (cross_term / (xtx_norm * yty_norm)).item()
    cka_reference = linear_cka(X, Y).item()

    print(f"\nCKA (recalculado a mano)     = {cka_manual:.6f}")
    print(f"CKA (modules.cka.linear_cka) = {cka_reference:.6f}")
```

```
    assert abs(cka_manual - cka_reference) < 1e-9, "no coinciden -- algo está␣
↪mal entendido arriba"
    print("(coinciden \u2713)")
    return cka_manual
```

1.2 Caso A — representaciones idénticas

$Y = X$. El caso trivial: si las dos representaciones son exactamente la misma matriz, CKA tiene que dar 1 — es el ancla para todo lo que sigue.

[3]:
```
X_a = np.array([
    [1.0, 2.0, 0.0],
    [3.0, 1.0, 1.0],
    [0.0, 0.0, 2.0],
    [2.0, 3.0, 1.0],
])
Y_a = X_a.copy()

_ = cka_step_by_step(X_a, Y_a, "A: Y = X (idénticas)")
```

```
=== A: Y = X (idénticas) ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)


X centrada:
 [[-0.5  0.5 -1. ]
 [ 1.5 -0.5  0. ]
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]
Y centrada:
 [[-0.5  0.5 -1. ]
 [ 1.5 -0.5  0. ]
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]


Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]

||Y_c^T X_c||_F^2 = 72.000000

X_c^T @ X_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||X_c^T X_c||_F = 8.485281
```

```
Y_c^T @ Y_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||Y_c^T Y_c||_F = 8.485281

CKA (recalculado a mano)     = 1.000000
CKA (modules.cka.linear_cka) = 1.000000
(coinciden )
```

1.3 Caso B — rotación pura

$Y = XR$ con R una matriz de rotación (ortogonal, $R^\top R = I$). CKA tiene que seguir dando 1: rotar las features no cambia las distancias/ángulos relativos entre ejemplos, y eso es lo único que CKA mira. Esta es la propiedad clave que la distingue de comparar pesos o activaciones coordenada-por-coordenada — por eso tiene sentido usarla entre dos modelos que nunca tuvieron por qué alinear sus cabezas de atención de la misma forma.

[4]:
```python
theta = np.radians(37)  # un ángulo arbitrario, nada especial
R = np.array([
    [np.cos(theta), -np.sin(theta), 0.0],
    [np.sin(theta),  np.cos(theta), 0.0],
    [0.0,            0.0,           1.0],
])
print("R^T R (debe ser ~identidad si R es ortogonal):\n", (R.T @ R).round(6))

Y_b = X_a @ R

_ = cka_step_by_step(X_a, Y_b, "B: Y = X @ R (rotación)")
```

```
R^T R (debe ser ~identidad si R es ortogonal):
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
=== B: Y = X @ R (rotación) ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)

X centrada:
 [[-0.5  0.5 -1. ]
 [ 1.5 -0.5  0. ]
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]
Y centrada:
 [[-0.098  0.7   -1.   ]
 [ 0.897 -1.302  0.   ]
 [-2.101 -0.295  1.   ]
```

```
 [ 1.302  0.897  0.   ]]

Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[ 5.197  4.606 -2.002]
 [-1.412  2.79  -0.995]
 [-1.    -2.     2.   ]]

||Y_c^T X_c||_F^2 = 72.000000

X_c^T @ X_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||X_c^T X_c||_F = 8.485281

Y_c^T @ Y_c:
[[ 6.923  0.551 -2.002]
 [ 0.551  3.077 -0.995]
 [-2.002 -0.995  2.   ]]
||Y_c^T Y_c||_F = 8.485281

CKA (recalculado a mano)     = 1.000000
CKA (modules.cka.linear_cka) = 1.000000
(coinciden )
```

1.4 Caso C — escala isotrópica

$Y = c \cdot X$, un mismo escalar c multiplicando TODAS las features por igual. CKA también da 1 — el numerador y el denominador de la fórmula crecen exactamente igual de rápido con c (por eso "isotrópica": la misma escala en todas las direcciones) y se cancelan. Este es el motivo por el que en el notebook de comparación de modelos no había que preocuparse por si un modelo aprendió activaciones "más grandes" que el otro en términos absolutos.

[5]:
```
c = 25.0
Y_c = c * X_a

_ = cka_step_by_step(X_a, Y_c, f"C: Y = {c} * X (escala isotrópica)")
```

```
=== C: Y = 25.0 * X (escala isotrópica) ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)

X centrada:
 [[-0.5  0.5 -1. ]
 [ 1.5 -0.5  0. ]
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]
Y centrada:
```

```
 [[-12.5  12.5 -25. ]
 [ 37.5 -12.5   0. ]
 [-37.5 -37.5  25. ]
 [ 12.5  37.5   0. ]]

Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[125.  50. -25.]
 [ 50. 125. -50.]
 [-25. -50.  50.]]

||Y_c^T X_c||_F^2 = 45000.000000

X_c^T @ X_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||X_c^T X_c||_F = 8.485281

Y_c^T @ Y_c:
[[ 3125.  1250.  -625.]
 [ 1250.  3125. -1250.]
 [ -625. -1250.  1250.]]
||Y_c^T Y_c||_F = 5303.300859

CKA (recalculado a mano)     = 1.000000
CKA (modules.cka.linear_cka) = 1.000000
(coinciden )
```

1.5 Caso D — permutar las features (columnas)

Y = las mismas columnas de X, en otro orden. Otra vez CKA = 1. Esto es lo que justifica comparar "cabeza I-2 contra cabeza NI-1" en el notebook anterior sin que importe el índice/orden interno de las cabezas: a CKA no le importa cuál columna es cuál, solo la estructura de similitud entre ejemplos que esas columnas generan en conjunto.

[6]:
```
Y_d = X_a[:, [2, 0, 1]]  # mismas columnas, orden distinto

_ = cka_step_by_step(X_a, Y_d, "D: Y = X con columnas permutadas")
```

```
=== D: Y = X con columnas permutadas ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)

X centrada:
 [[-0.5  0.5 -1. ]
 [ 1.5 -0.5  0. ]
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]
```

```
Y centrada:
 [[-1.  -0.5  0.5]
 [ 0.   1.5 -0.5]
 [ 1.  -1.5 -1.5]
 [ 0.   0.5  1.5]]

Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[-1. -2.  2.]
 [ 5.  2. -1.]
 [ 2.  5. -2.]]

||Y_c^T X_c||_F^2 = 72.000000

X_c^T @ X_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||X_c^T X_c||_F = 8.485281

Y_c^T @ Y_c:
[[ 2. -1. -2.]
 [-1.  5.  2.]
 [-2.  2.  5.]]
||Y_c^T Y_c||_F = 8.485281

CKA (recalculado a mano)     = 1.000000
CKA (modules.cka.linear_cka) = 1.000000
(coinciden )
```

1.6 Caso E — escala NO isotrópica (rompe la invariancia)

Ahora estiramos SOLO una feature mucho más que las otras. Esto ya no es una rotación ni una escala uniforme, así que CKA debería BAJAR de 1 — es la prueba de que la invariancia de CKA tiene un límite real, no es "cualquier transformación lineal da 1".

[7]:
```
Y_e = X_a.copy()
Y_e[:, 0] *= 50.0  # solo la primera feature se dispara

_ = cka_step_by_step(X_a, Y_e, "E: Y = X con una sola feature escalada x50 (NO␣
 ↪isotrópica)")
```

```
=== E: Y = X con una sola feature escalada x50 (NO isotrópica) ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)

X centrada:
 [[-0.5  0.5 -1. ]
 [ 1.5 -0.5  0. ]
```

```
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]
Y centrada:
 [[-25.    0.5  -1. ]
 [ 75.   -0.5   0. ]
 [-75.   -1.5   1. ]
 [ 25.    1.5   0. ]]

Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[250. 100. -50.]
 [  2.   5.  -2.]
 [ -1.  -2.   2.]]

||Y_c^T X_c||_F^2 = 75042.000000

X_c^T @ X_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||X_c^T X_c||_F = 8.485281

Y_c^T @ Y_c:
[[12500.   100.   -50.]
 [  100.     5.    -2.]
 [  -50.    -2.     2.]]
||Y_c^T Y_c||_F = 12501.001440

CKA (recalculado a mano)     = 0.707446
CKA (modules.cka.linear_cka) = 0.707446
(coinciden )
```

1.7 Caso F — ruido: parecidas pero no iguales

$Y = X + \varepsilon$, ruido pequeño. Debería dar un número alto pero ya no exactamente 1 — el caso realista de "dos representaciones que capturan básicamente lo mismo pero no perfectamente".

[8]:
```
rng = np.random.default_rng(0)
noise = rng.normal(scale=0.3, size=X_a.shape)
Y_f = X_a + noise

_ = cka_step_by_step(X_a, Y_f, "F: Y = X + ruido pequeño")
```

```
=== F: Y = X + ruido pequeño ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)

X centrada:
 [[-0.5  0.5 -1. ]
```

```
 [ 1.5 -0.5  0. ]
 [-1.5 -1.5  1. ]
 [ 0.5  1.5  0. ]]
Y centrada:
 [[-0.482  0.486 -0.833]
 [ 1.511 -0.635  0.083]
 [-1.129 -1.19   0.763]
 [ 0.1    1.339 -0.013]]

Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[ 4.252  0.847 -0.647]
 [ 1.259  4.354 -1.676]
 [-0.61  -1.623  1.597]]

||Y_c^T X_c||_F^2 = 48.119835

X_c^T @ X_c:
[[ 5.  2. -1.]
 [ 2.  5. -2.]
 [-1. -2.  2.]]
||X_c^T X_c||_F = 8.485281

Y_c^T @ Y_c:
[[ 3.801  0.284 -0.336]
 [ 0.284  3.848 -1.384]
 [-0.336 -1.384  1.284]]
||Y_c^T Y_c||_F = 5.926631

CKA (recalculado a mano)     = 0.956863
CKA (modules.cka.linear_cka) = 0.956863
(coinciden )
```

1.8 Caso G — completamente independientes

X y Y sin ninguna relación (dos matrices aleatorias distintas). CKA debería caer bastante más cerca de 0 que los casos anteriores — aunque con tan pocos ejemplos ($n = 4$) nunca llega exactamente a 0 por azar; con más ejemplos se acercaría más. Vale la pena compararlo contra los números reales entre cabezas del notebook anterior (0.05–0.3 típicamente) para calibrar qué tan "bajo es bajo".

[9]:
```
X_g = rng.normal(size=(4, 3))
Y_g = rng.normal(size=(4, 3))

_ = cka_step_by_step(X_g, Y_g, "G: X, Y independientes (ruido puro, sin␣
 ↪relación)")
```

```
=== G: X, Y independientes (ruido puro, sin relación) ===
X: (4, 3) (n=4 ejemplos, p1=3 features)
Y: (4, 3) (n=4 ejemplos, p2=3 features)
```

```
X centrada:
 [[-2.005 -0.122 -0.911]
 [-0.412 -0.448  0.019]
 [ 0.731  1.139  0.206]
 [ 1.686 -0.569  0.686]]
Y centrada:
 [[ 1.025  0.184 -0.662]
 [-0.8   -0.368  0.302]
 [-0.888 -0.12  -0.077]
 [ 0.663  0.304  0.437]]

Y_c^T @ X_c  (shape (3, 3) = p2 x p1):
 [[-1.258 -1.155 -0.677]
 [ 0.209 -0.167  0.01 ]
 [ 1.883 -0.391  0.893]]

||Y_c^T X_c||_F^2 = 7.942265

X_c^T @ X_c:
[[ 7.57   0.304  3.128]
 [ 0.304  1.836 -0.052]
 [ 3.128 -0.052  1.344]]
||X_c^T X_c||_F = 9.068044

Y_c^T @ Y_c:
[[ 2.918  0.791 -0.562]
 [ 0.791  0.276 -0.09 ]
 [-0.562 -0.09   0.726]]
||Y_c^T Y_c||_F = 3.319236

CKA (recalculado a mano)     = 0.263872
CKA (modules.cka.linear_cka) = 0.263872
(coinciden )
```

1.9 Resumen — la regla de oro

Caso	Transformación de X a Y	CKA
A	idéntica	1.000
B	rotación ortogonal	1.000
C	escala isotrópica ($c \cdot X$)	1.000
D	permutar columnas	1.000
E	escala NO isotrópica (una feature x50)	baja
F	+ ruido pequeño	alto pero < 1
G	independientes	cerca de 0 (con pocos ejemplos, no exactamente)

La intuición que queda: CKA compara la matriz de similitud entre EJEMPLOS que induce cada

representación ($XX^\top$ vs. $YY^\top$, en el fondo), no los valores de las features en sí. Por eso es ciega a rotar, reflejar, permutar o reescalar uniformemente las features — pero si una transformación distorsiona esa geometría relativa entre ejemplos (como estirar una sola feature), CKA sí lo nota y el número baja. Esa es la vara con la que hay que leer los números del notebook de comparación interpretable vs. no interpretable: un CKA bajo entre dos cabezas no dice "una está mal", dice "la geometría relativa entre estos 15 ejemplos, según esta cabeza, no se parece a la que arma esa otra cabeza".